Mocking API Calls, Hooks & Context

4 min read Updated 24 Mar 2026 Topic 42 of 62

Bookmark Mark done

9.3 – Mocking API Calls, Hooks & Context

🎯 What Interviewers Actually Ask

Question	Level	Frequency
"How do you mock API calls in tests?"	🟡🔴	★★★★☆
"How do you test a component that uses Context?"	🔴	★★★☆☆
"How do you test custom hooks?"	🔴	★★★☆☆

Mocking API Calls with Jest

The simplest approach is to mock the `fetch` function globally.

jsx

JSX Copy

```
});

afterEach(() => {
  jest.restoreAllMocks();
});

test('displays users from API', async () => {
  global.fetch.mockResolvedValueOnce({
    ok: true,
    json: async () => [
      { id: 1, name: 'Alice' },
      { id: 2, name: 'Bob' },
    ],
  });

  render(<UserList />);

  expect(await screen.findByText('Alice')).toBeInTheDocument();
  expect(screen.getByText('Bob')).toBeInTheDocument();
  expect(global.fetch).toHaveBeenCalledWith('/api/users');
});

test('displays error message on API failure', async () => {
  global.fetch.mockRejectedValueOnce(new Error('Network error'));

  render(<UserList />);

  expect(await screen.findByText(/error/i)).toBeInTheDocument();
});
```

Mock Service Worker (MSW) for realistic mocking

MSW intercepts requests at the network level, making components behave exactly as in production.

jsx

JSX Copy

```
const server = setupServer(
  rest.get('/api/users', (req, res, ctx) => {
    return res(ctx.json([
      { id: 1, name: 'Alice' },
      { id: 2, name: 'Bob' },
    ]));
  })
);

beforeAll(() => server.listen());
afterEach(() => server.resetHandlers());
afterAll(() => server.close());

test('loads users', async () => {
  render(<UserList />);
  expect(await screen.findByText('Alice')).toBeInTheDocument();
});

test('handles server error', async () => {
  server.use(
    rest.get('/api/users', (req, res, ctx) => {
      return res(ctx.status(500));
    })
  );

  render(<UserList />);
  expect(await screen.findByText(/error/i)).toBeInTheDocument();
});
```

Testing Components with Context

Wrap the component in the Provider during the test, supplying the values you want.

jsx

JSX Copy

```
    <AuthContext.Provider value={{ user, login: jest.fn(), logout: jest.fn
      {ui}
    </AuthContext.Provider>
  );
}

test('shows welcome message when logged in', () => {
  renderWithAuth(<Header />, { user: { name: 'Alice' } });
  expect(screen.getByText('Welcome, Alice')).toBeInTheDocument();
});

test('shows login button when not logged in', () => {
  renderWithAuth(<Header />);
  expect(screen.getByRole('button', { name: /login/i })).toBeInTheDocument
});
```

Creating a custom `renderWith ...` helper is the standard pattern. It keeps tests DRY and readable.

Testing Custom Hooks

Use `renderHook` from `@testing-library/react` to test hooks outside of a component.

jsx

JSX Copy

```
import { renderHook, act } from '@testing-library/react';
import { useCounter } from './useCounter';

test('increments the counter', () => {
  const { result } = renderHook(() => useCounter());

  expect(result.current.count).toBe(0);
```

```
  expect(result.current.count).toBe(1);
});

test('starts with initial value', () => {
  const { result } = renderHook(() => useCounter(10));
  expect(result.current.count).toBe(10);
});
```

Wrap state-changing actions in `act()` to ensure React processes the update before your assertions run.

What to Test (and What Not To)

Test this ✅	Skip this ❌
User-facing behavior	Internal state values
Rendered output	Implementation details (which hook was called)
Form validation messages	CSS classes or styles (unless visually critical)
API loading/error/success states	Third-party library internals
Keyboard and mouse interactions	Whether a console.log fired

Interview Answer

> "For API calls, I mock fetch directly for simple cases or use MSW for more realistic network-level mocking. For components that depend on Context, I create custom render helpers that wrap the component in the necessary Providers. For custom hooks, I use renderHook from RTL. I prioritize testing the happy path, error states, and edge cases like empty lists and loading states."

Key Takeaways for Module 9

- **Testing pyramid:** Many unit tests, fewer integration tests, very few E2E tests.
- **RTL philosophy:** Test what users see and do, not implementation details.
- **Query priority:** `getByRole` first, `getByTestId` last. Use `queryBy` for "not found" assertions, `findBy` for async.
- **userEvent** is more realistic than `fireEvent`.
- **Mock API calls** with `jest.fn()` on fetch, or MSW for network-level interception.
- **Test Context** by wrapping components in Providers inside custom render helpers.
- **Test hooks** with `renderHook` and wrap state changes in `act()`.
- **Snapshots** should be used sparingly, only for stable, static components.

QUICK CHECK — no marks, just for you.

1. [illegible] that reads from AuthContext. According to the lesson, what is the standard way to control what the component sees during the test?

- Render the component normally and use act() to push the context value in afterwards
- Wrap the component in the Context Provider inside a custom renderWith... helper and pass the values you want
- Export a setter from the context module and call it before render to seed the value
- Call jest.mock on the React useContext export so it returns your fake value

Check answer

2. In the renderHook example, result.current.increment() is called inside act(). What does act() accomplish here?

- It makes the hook run outside a component, which renderHook cannot do on its own
- It ensures React processes the state update before the following assertions run
- It converts the synchronous increment call into an awaitable promise
- It resets result.current back to the hook's initial value between assertions

Check answer

3. The lesson contrasts mocking fetch with jest.fn() against using Mock Service Worker. What specifically distinguishes the MSW approach?

- MSW intercepts requests at the network level, so the component behaves exactly as it does in production
- MSW replaces the global fetch reference, so no beforeEach or afterEach cleanup is ever required

MSW asserts on request URLs automatically, removing the need for toHaveBeenCalledWith checks

MSW skips the component's real data-fetching code and injects the response into state directly

Check answer

Previous
React Testing Library: Practical Guide

Next
Scenario: "This React App is Slow. How Do You Fix It?"

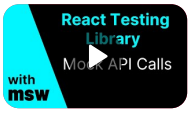
LEARN MORE

ARTICLES & DOCS

Mock Service Worker (MSW)
mswjs.io

React Testing Library — intro
testing-library.com

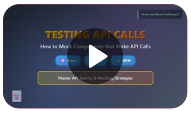
VIDEOS



Mock API calls with MSW
YouTube



Mock a REST API with Jest & RTL
YouTube



Mocking API calls (Jest & MSW)
YouTube

Hand-picked references for this lesson.